RAPPORT DE STAGE

Migration du Datamart Construction de SAS vers PySpark

Pour la validation du stage de fin d'études

INFORMATIONS

Auteur : O. A. Stephane KPOVIESSI
Formation : BIG DATA — Promotion 2025

Entreprise d'accueil : Allianz France
Direction P4D — Pôle Advanced Data & Climate

Encadrant entreprise : M. ROMEO TOHOUN
Enseignant référent : Madame Feryal WINDAL

JUNIA ISEN Lille
41 boulevard Vauban
59000 Lille

Avril 2025

Allianz France
20 Pl. de Seine
92400 Courbevoie

Résumé

Ce rapport présente la migration du datamart Construction d'Allianz France, historiquement développé en SAS, vers une architecture Python/PySpark sur Azure Databricks. Ce projet s'inscrit dans le programme SAS EXIST, initiative stratégique visant à moderniser progressivement l'ensemble des datamarts P&C de l'entreprise.

La démarche a débuté par une analyse exhaustive du code SAS existant, dix-neuf fichiers, environ quinze mille lignes, sans documentation initiale ayant conduit à la production d'une documentation fonctionnelle complète. Sur cette base, une architecture médaillon (Bronze/Silver/Gold) a été conçue et validée comme standard pour les prochaines migrations. L'ensemble des règles de gestion (72) et des sources de données (43) a été recensé dans deux fichiers Excel de traçabilité avant tout développement.

Trois pipelines Python ont été implémentés avec PySpark : gestion du portefeuille et des mouvements (PTF Mouvements), calcul des capitaux avec indexation FFB (Capitaux), et agrégation des primes (Émissions). La validation repose sur la comparaison des KPI entre les sorties SAS de référence et les sorties Python, sur dix visions mensuelles. La parité fonctionnelle est confirmée pour les pipelines PTF Mouvements et Capitaux. La validation des Émissions est en cours.

Ce prototype établit les fondations (code, documentation, méthodologie) pour la migration des quatre marchés restants (Auto, Habitation, Santé, Épargne). Les modules développés (lecteurs génériques, système de logging, configuration externalisée) sont directement réutilisables, réduisant significativement l'investissement nécessaire pour les migrations suivantes.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réussite de mon stage au sein d'Allianz France et à l'élaboration de ce rapport.

Mes premiers remerciements s'adressent à **Monsieur TOHOUN ROMEO**, mon maître de stage au sein du pôle Advanced Data & Climate de la direction P4D, pour son accueil, sa disponibilité et la confiance qu'il m'a accordée tout au long de ces six mois. Son encadrement bienveillant, ses conseils avisés et son expertise technique m'ont permis de progresser considérablement et de mener à bien ce projet de migration. Sa pédagogie et son partage d'expérience ont été déterminants dans ma compréhension des enjeux métier et techniques.

Je remercie également **Madame Caroline BOUCHON** pour son suivi régulier, ses recommandations stratégiques et le temps consacré aux points d'avancement qui ont éclairé mes choix tout au long du projet.

Mes remerciements s'adressent aussi à l'ensemble de **l'équipe du pôle Advanced Data & Climate** pour leur accueil chaleureux, leur bienveillance et les moments d'échange enrichissants que nous avons partagés. Je pense particulièrement à **M. VOSS Alain**, notre expert en assurance, pour ses explications détaillées sur les mécanismes assurantiels qui m'ont permis de lever les ambiguïtés métier, et à tous mes collègues pour leur disponibilité et leur aide précieuse dans la compréhension du fonctionnement de l'équipe et des projets transverses.

Je souhaite exprimer ma reconnaissance envers **Allianz France**, et plus particulièrement la **direction P4D**, pour m'avoir offert l'opportunité de réaliser ce stage au cœur d'un projet stratégique de modernisation des infrastructures de données. Cette expérience m'a permis de mettre en pratique mes connaissances académiques sur un projet concret d'envergure et de découvrir le fonctionnement d'une grande entreprise engagée dans la transformation digitale.

Enfin, je tiens à remercier **JUNIA ISEN Lille** et l'ensemble du corps professoral pour la qualité de la formation dispensée, qui m'a fourni les bases techniques et méthodologiques nécessaires pour aborder ce stage avec confiance. La solidité de cet enseignement m'a permis d'appréhender efficacement les défis techniques rencontrés lors de cette migration.

Lexique

Ce lexique regroupe les termes métier, acronymes internes et concepts spécifiques au projet utilisés dans ce rapport.

Termes métier assurance

AFN (Affaires Nouvelles) : Polices souscrites ou renouvelées au cours de la période. Indicateur de production commerciale.

AZ : Canal de distribution regroupant les Agents et le Courtage traditionnel. Se distingue d'AZEC par ses sources de données et sa logique métier.

AZEC : Canal de Courtage Construction (entreprises de construction). Sources de données et règles de gestion distinctes du canal AZ.

Coassurance : Partage d'un risque entre plusieurs assureurs. La compagnie ne porte qu'une fraction du risque total (en pourcentage).

LCI (Limite Contractuelle d'Indemnité) : Montant maximum que l'assureur s'engage à verser en cas de sinistre.

P&C (Property & Casualty) : Assurance des biens et responsabilités (Construction, Auto, Habitation, Santé, Épargne).

PE (Perte d'Exploitation) : Capital assuré couvrant les pertes financières consécutives à un sinistre.

PTF (Portefeuille) : Ensemble des polices en vigueur à une date donnée.

RD (Risque Direct) : Capital assuré pour les dommages matériels directs.

RES (Résiliations) : Polices résiliées ou non renouvelées au cours de la période.

RPC (Remises en Portefeuille Avec Modification de Prime) : Polices réintégrées avec ajustement tarifaire.

RPT (Remises en Portefeuille Sans Modification de Prime) : Polices réintégrées sans changement de prime.

SMP (Sinistre Maximum Possible) : Estimation du coût maximum d'un sinistre pour l'assureur.

Acronymes internes Allianz

ADC (Advanced Data & Climate) : Pôle d'Allianz France spécialisé en data management et modélisations. Équipe d'accueil du stage.

CDPOLE : Code Pôle de distribution : « 1 » = Agents, « 3 » = Courtage.

DIRCOM : Direction commerciale / canal de distribution (« AZ » ou « AZEC »).

IRD (Inventaire des Risques et Données) : Référentiel Allianz décrivant les caractéristiques techniques des risques assurés.

P4D : Direction d'Allianz France regroupant plusieurs pôles data, dont ADC.

SAS EXIST : Programme stratégique de migration des datamarts SAS vers Azure/PySpark.

Sommaire

Résumé **1**

Remerciements **2**

Lexique **3**

Sommaire **4**

Introduction **6**

1 Cadre de l'étude **7**
1.1 Présentation d'Allianz France 7
1.1.1 Le groupe Allianz : un acteur mondial de l'assurance 7
1.1.2 Allianz France : un positionnement fort 7
1.2 La direction P4D et le pôle Advanced Data & Climate 8
1.2.1 La direction Pricing, Portfolio, Pilotage, Performance & Data (P4D) . . . 8
1.3 Contexte stratégique : le projet SAS EXIST 8
1.4 Problématique et objectifs de la mission 9

2 Contexte technique et état de l'art **9**
2.1 L'existant : le datamart Construction en SAS 9
2.1.1 Rôle du datamart Construction dans l'analyse assurantielle 9
2.1.2 Architecture actuelle en SAS 10
2.1.3 Limites de l'architecture SAS existante 12
2.2 Technologies modernes pour le traitement de données 12
2.2.1 PySpark et Apache Spark 12
2.2.2 Azure Databricks 12
2.2.3 Formats de stockage modernes 13
2.2.4 L'architecture médaillon : standard de l'industrie 13
2.3 Revue bibliographique et état de l'art des migrations SAS vers Python 14
2.3.1 Tendances de migration dans l'industrie 14
2.3.2 Défis techniques connus 15
2.3.3 Bonnes pratiques identifiées 15

3 Méthodologie et moyens **15**
3.1 Démarche globale et organisation du projet 15
3.1.1 Une approche itérative et structurée 15
3.1.2 Équipe et cadre de travail 16
3.2 Phase 1 : Analyse et documentation du code SAS existant 16
3.2.1 Objectifs et situation initiale 16
3.2.2 Démarche d'analyse systématique 16
3.2.3 Livrables de la phase 1 17
3.3 Phase 2 : Conception de l'architecture cible Python 17

3.3.1 Évaluation des options architecturales . . . 17
3.3.2 Principes de conception retenus . . . 17
3.3.3 Stack technologique validée . . . 18
3.4 Phase 3 : Recensement systématique des règles et datasets . . . 18
3.4.1 Objectif : tout tracer avant de coder . . . 18
3.4.2 Fichier 1 : inventaire des règles de gestion . . . 18
3.4.3 Fichier 2 : recensement des datasets sources . . . 18
3.5 Phase 4 : Implémentation des pipelines Python . . . 19
3.5.1 Structure du projet . . . 19
3.5.2 Ordre d'implémentation et stratégie de développement . . . 19
3.5.3 Architecture des processors . . . 20
3.6 Phase 5 : Validation et tests de parité fonctionnelle . . . 20
3.6.1 Stratégie de validation à trois niveaux . . . 20
3.6.2 Défis techniques rencontrés lors de la validation . . . 20
3.7 Outils, environnement et livrables finaux . . . 22
3.7.1 Environnement de développement . . . 22
3.7.2 Synthèse des livrables produits . . . 22

4 PRÉSENTATION ET ANALYSE DES RÉSULTATS . . . 23
4.1 Architecture finale implémentée . . . 23
4.2 Résultats par pipeline . . . 23
4.2.1 Pipeline PTF Mouvements . . . 23
4.2.2 Pipeline Capitaux . . . 24
4.2.3 Pipeline Émissions . . . 24
4.3 Validation de la parité fonctionnelle . . . 24
4.4 Analyse de la structure du code Python produit . . . 25
4.4.1 Modularité et réutilisabilité . . . 25
4.4.2 Configuration externalisée . . . 25
4.4.3 Comparaison qualitative SAS vs Python . . . 25

5 DISCUSSION ET PERSPECTIVES . . . 25
5.1 Analyse critique du projet . . . 25
5.2 Positionnement par rapport aux pratiques de l'industrie . . . 26
5.3 Ce que ça apporte à Allianz France . . . 26
5.4 Perspectives . . . 26

Conclusion . . . 28

Bibliographie . . . 29

Introduction

La transformation numérique des entreprises s'est imposée comme un enjeu stratégique majeur depuis le début des années 2010. Toutes les grandes organisations ont dû revoir leurs systèmes d'information, leurs infrastructures de données et leurs méthodes de travail pour faire face à l'accélération technologique et à la multiplication des sources. Dans le secteur de l'assurance, cet impératif est renforcé par la pression réglementaire, la concurrence et la nécessité de traiter des volumes de données croissants avec précision et rapidité.

C'est dans ce contexte qu'Allianz France, l'une des principales compagnies d'assurance en France, a entrepris un programme de modernisation de ses systèmes de traitement de données. Historiquement, les datamarts P&C (Property & Casualty) de l'entreprise étaient développés sous SAS, un logiciel propriétaire largement répandu dans les grandes institutions financières et assurantielles. Face aux limites de cette technologie (coûts de licences élevés, performances insuffisantes pour les volumes actuels et expertise de plus en plus rare) et à l'émergence de solutions open source performantes, Allianz France a lancé le projet **SAS EXIST** (SAS Exit Strategy), dont l'objectif est la migration progressive de ses datamarts vers une architecture Python/PySpark sur Azure Databricks.

C'est dans le cadre de ce projet stratégique que s'inscrit ce stage de six mois au sein du pôle Advanced Data & Climate (ADC) de la direction P4D d'Allianz France. La mission confiée consiste à réaliser le premier prototype de cette migration, ciblant le datamart **Construction** (le marché de l'assurance construction), sélectionné pour sa représentativité et sa complexité maîtrisable en tant que cas pilote. Ce travail vise deux objectifs complémentaires : démontrer la faisabilité technique de la migration en assurant la parité fonctionnelle avec les sorties SAS de référence, et établir un cadre méthodologique et documentaire réutilisable pour les migrations à venir.

Ce rapport retrace l'ensemble du travail accompli durant ce stage. Il est organisé en cinq chapitres. Le premier situe le contexte de l'entreprise et du projet SAS EXIST. Le deuxième décrit les systèmes SAS existants et les technologies cibles retenues. Le troisième présente la méthodologie adoptée, articulée en cinq phases. Le quatrième analyse les résultats obtenus pipeline par pipeline. Le cinquième propose une réflexion critique sur le projet et ouvre sur les perspectives pour la suite du programme.

Chapitre 1 . Cadre de l'étude

1.1 Présentation d'Allianz France

1.1.1 Le groupe Allianz : un acteur mondial de l'assurance

Le groupe Allianz a été fondé en 1890 à Berlin et son siège social se trouve aujourd'hui à Munich. Avec plus de 130 ans d'existence, Allianz s'est imposé comme l'un des leaders mondiaux de l'assurance et de la gestion d'actifs. Le groupe est présent dans plus de 70 pays à travers le monde, couvrant l'ensemble des continents : Europe, Amérique du Nord, Europe centrale et orientale, et la région Asie-Pacifique.

L'offre d'Allianz couvre l'ensemble des besoins assurantiels : assurances dommages (automobile, habitation, construction), assurances santé, prévoyance et assurances vie. Le groupe s'adresse aussi bien aux particuliers qu'aux professionnels, entreprises et collectivités. Au-delà de l'assurance, Allianz propose également des produits bancaires au travers de son réseau d'agents, diversifiant ainsi ses activités financières.

1.1.2 Allianz France : un positionnement fort

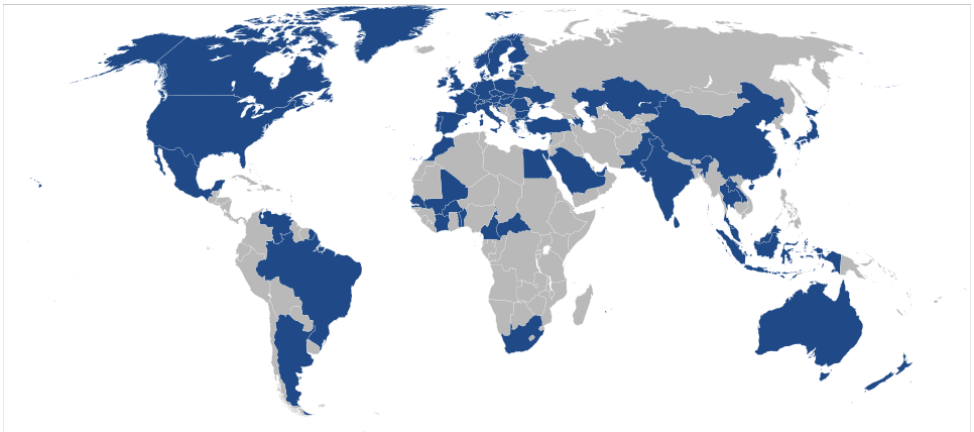


FIGURE 1.1 – Implantation mondiale du groupe Allianz (Source : Wikimedia Commons, consulté le fév. 2026)

Aujourd'hui, Allianz France, dont le siège est situé à Paris, est un fournisseur majeur de services financiers spécialisé dans l'assurance et la gestion d'actifs. Avec près de 2 500 points de vente répartis sur le territoire, elle dispose du deuxième plus grand réseau français. L'entreprise emploie environ 11 000 collaborateurs et accompagne plus de 5 millions de clients partout en France. Son offre est particulièrement diversifiée et couvre tous les besoins de la vie quotidienne : assurance automobile, bateau, habitation, construction, santé, prévoyance, assurance vie, retraite, services bancaires et protection juridique.

Les résultats financiers du groupe témoignent de sa solidité. En 2019, Allianz France affichait un chiffre d'affaires compris entre 11 et 12 milliards d'euros, confirmant sa position de leader sur le marché français. Cette performance repose sur la qualité de ses services, son réseau étendu et la diversité de son portefeuille d'activités.

Allianz France s'appuie sur six filiales spécialisées : assurance des grands risques industriels (AGCS), gestion d'actifs (Allianz Global Investors), immobilier (Allianz Real Estate), assurance voyage et assistance (Allianz Global Assistance), et assurance-crédit (Euler Hermes).

1.2 La direction P4D et le pôle Advanced Data & Climate

1.2.1 La direction Pricing, Portfolio, Pilotage, Performance & Data (P4D)

Au sein de l'unité Assurances de Biens et de Responsabilités (ABR) d'Allianz France, la direction P4D joue un rôle central dans la définition de la stratégie tarifaire, le pilotage du portefeuille et la gestion des données. Comme son nom l'indique, P4D regroupe quatre dimensions essentielles : le Pricing (tarification), le Portfolio (gestion de portefeuille), le Pilotage (suivi des performances) et la Performance & Data (exploitation des données).

Cette direction structure ses activités autour de plusieurs pôles spécialisés, chacun dédié à un segment de marché ou à une expertise transverse. On y trouve notamment les pôles Pricing & Portfolio pour les différents marchés (Auto, IRD Particuliers, Pros, MidCorp), ainsi que deux pôles transverses : Business & Strategic Steering, et Advanced Data & Climate. C'est au sein de ce dernier qu'a été réalisée la mission présentée dans ce rapport.

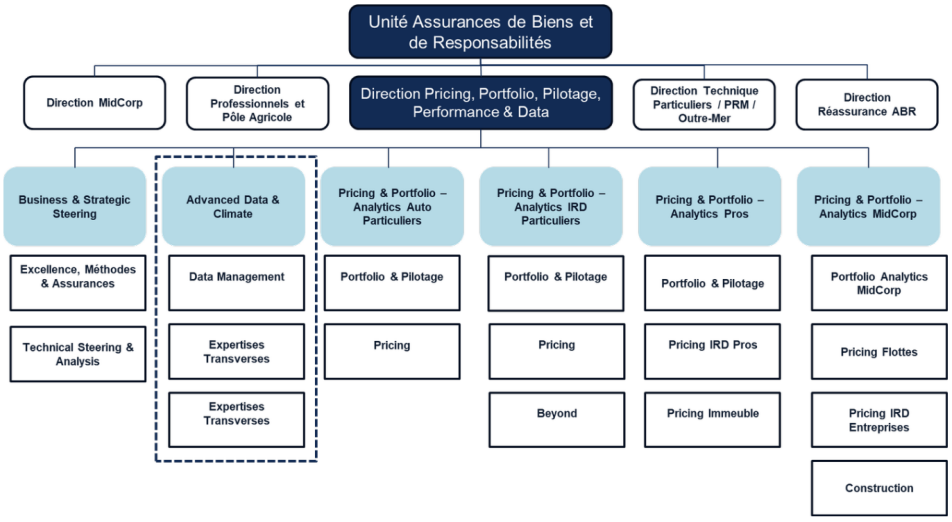


FIGURE 1.2 – Organigramme de l'unité ABR, direction P4D et pôle Advanced Data & Climate

Le pôle ADC regroupe trois activités complémentaires :

— **Data Management** : mise à disposition et maintenance des datamarts par marché.

— **Expertises Transverses** : modélisations actuarielles et statistiques.

— **Expertises Climatiques** : modèles de risque liés au changement climatique.

La mission s'est déroulée au sein de l'équipe Data Management, responsable des datamarts Construction, Auto, Habitation, Santé et Épargne qui alimentent les analyses de l'ensemble de la direction P4D.

1.3 Contexte stratégique : le projet SAS EXIST

Tous les datamarts P&C d'Allianz France fonctionnent aujourd'hui sur une infrastructure SAS en place depuis de nombreuses années. Ce système a bien servi l'entreprise, mais il présente désormais des limites difficiles à ignorer : coûts de licences propriétaires élevés, performances contraintes par des serveurs vieillissants, code difficile à maintenir faute de documentation, et expertise SAS de plus en plus rare sur le marché du travail. Pendant ce temps, Python et l'écosystème cloud sont devenus les standards du data engineering.

C'est dans ce contexte qu'Allianz France a lancé en 2024 le projet **SAS EXIST** (SAS Exit Strategy) : une migration progressive de l'ensemble des datamarts vers Azure Databricks et

PySpark. La migration se fait marché par marché, en commençant par un prototype pilote qui devra valider la démarche technique et poser les bases méthodologiques pour les migrations suivantes.

1.4 Problématique et objectifs de la mission

La mission confiée consistait à réaliser ce premier prototype sur le **datamart Construction** (marché 6), choisi pour sa complexité représentative sans être excessive. La problématique centrale était la suivante :

> *Comment migrer le datamart Construction de SAS vers PySpark en garantissant une parité fonctionnelle stricte et en construisant une architecture réutilisable pour les migrations futures ?*

Cette problématique engage quatre dimensions :

- **Technique** : analyser et transposer près de 15 000 lignes de code SAS réparties dans dix-neuf fichiers hétérogènes, pour la plupart non documentés et comportant de nombreuses dépendances implicites ;
- **Architecturale** : concevoir une architecture python adaptée aux spécificités du marché Construction et généralisable aux autres marchés ;
- **Qualité** : valider que les sorties Python sont strictement identiques aux sorties SAS de référence, sur plusieurs périodes ;
- **Documentation** : produire des livrables durables, documentation SAS + python, règles métier, code Python, qui serviront de référence aux équipes après le stage.

Le succès de ce prototype conditionne directement la suite du programme SAS EXIST : si la faisabilité est démontrée, les quatre marchés restants pourront être migrés selon la même méthodologie.

Chapitre 2 . Contexte technique et état de l'art

2.1 L'existant : le datamart Construction en SAS

2.1.1 Rôle du datamart Construction dans l'analyse assurantielle

Le datamart Construction constitue l'un des cinq datamarts P&C gérés par l'équipe Data Management d'Allianz France. Il agrège et structure l'ensemble des données relatives au marché de l'assurance construction, qui couvre les risques liés aux bâtiments, travaux et chantiers. Ce marché présente des spécificités métier importantes : les capitaux assurés sont généralement élevés, les sinistres peuvent être complexes et s'étaler sur plusieurs années, et l'indexation des capitaux en fonction de l'inflation dans le secteur de la construction est cruciale.

Le datamart Construction alimente trois types d'analyses principales. Les **études de portefeuille** permettent de suivre l'évolution du nombre de polices, les mouvements d'affaires nouvelles et de résiliations, et la composition du portefeuille par segment de clientèle (agents, courtiers, entreprises de construction). Les **analyses de capitaux** portent sur l'évaluation de l'exposition au risque à travers les capitaux assurés (Sinistre Maximum Possible, Limite Contractuelle d'Indemnité, Perte d'Exploitation, Risque Direct) et leur indexation selon les indices de coût de construction de la Fédération Française du Bâtiment. Enfin, les **calculs de primes** agrègent

les émissions de primes par police et par garantie pour alimenter les reportings financiers et réglementaires.

Les utilisateurs principaux de ce datamart sont les actuaires de la direction P4D, les équipes de pricing qui calibrent les tarifs commerciaux, les contrôleurs de gestion qui suivent la rentabilité du marché, et les équipes de pilotage qui produisent les reportings pour la direction générale et les autorités de régulation. La disponibilité et la fiabilité de ce datamart sont donc critiques pour le fonctionnement quotidien de l'entreprise.

La figure 2.1 ci-dessous illustre la vue d'ensemble du datamart Construction en SAS : les trois sources de données principales (IMS pour le portefeuille AZ et AZEC, OneBI pour les émissions, et les référentiels métier) alimentent trois pipelines distincts produisant trois sorties analytiques finaux.

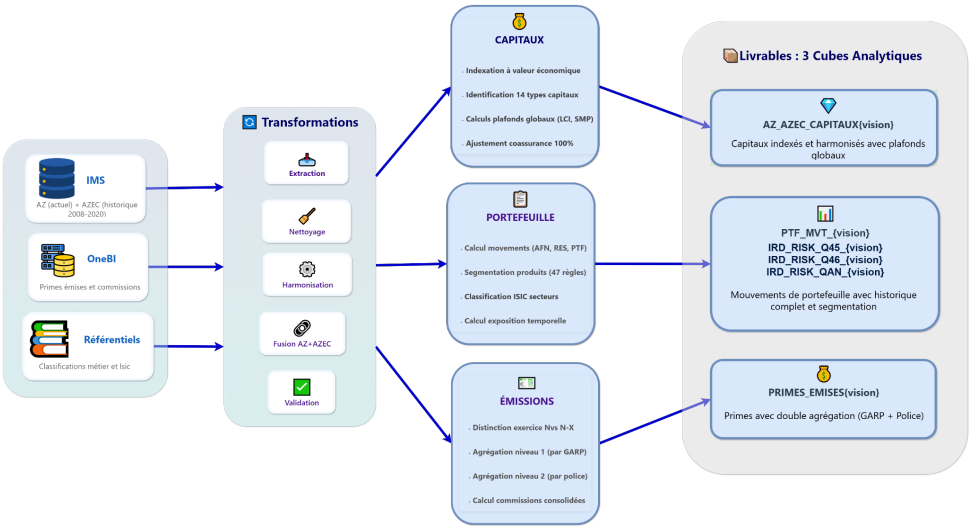


FIGURE 2.1 – Vue d'ensemble de l'architecture du datamart Construction : sources de données, transformations et sorties analytiques produits

2.1.2 Architecture actuelle en SAS

L'implémentation actuelle du datamart Construction repose sur une architecture SAS développée progressivement sur plusieurs années. Le code est réparti sur **dix-neuf fichiers SAS** totalisant environ quinze mille lignes de code. Ces fichiers sont organisés selon une logique séquentielle : chaque fichier réalise une étape de transformation spécifique et produit des tables intermédiaires qui servent d'entrée au fichier suivant.

L'architecture SAS suit un modèle monolithique où toutes les transformations sont enchaînées dans un flux linéaire. Les données brutes, stockées sous forme de fichiers CSV sur des serveurs partagés, sont lues par les premiers fichiers SAS qui appliquent des filtres métier de base (marché Construction, exclusion des polices annulées). Les fichiers suivants enrichissent progressivement ces données : extraction des capitaux depuis des champs texte, calculs des mouvements de portefeuille, jointures avec des tables de référence (nomenclatures NAF, codes ISIC, données IRD), calculs d'indexation, et enfin agrégation pour produire les tables finales.

Comme le montre la figure 2.2, l'architecture détaillée du datamart décompose les traitements en trois pipelines indépendants. Le **pipeline Capitaux** (partie gauche) extrait les capitaux historiques et actuels, applique l'indexation à valeur économique par rapport aux indices FFB, puis consolide les données AZ et AZEC en un sortie final. Le **pipeline Portefeuille et Mouvements** (partie centrale) réalise l'extraction des données de portefeuille, le nettoyage et l'harmonisation des structures AZ/AZEC, la fusion intelligente des deux canaux de distribution, l'enrichissement métier (segmentation produits, indicateurs AFN/RES, codes ISIC), et

enfin l'enrichissement DataRisk pour produire les sorties PTF finaux. Le **pipeline Émissions** (partie droite) centralise les données One BI, applique un nettoyage et une validation, réalise la segmentation et la classification produits, et agrège les primes par police et par garantie.

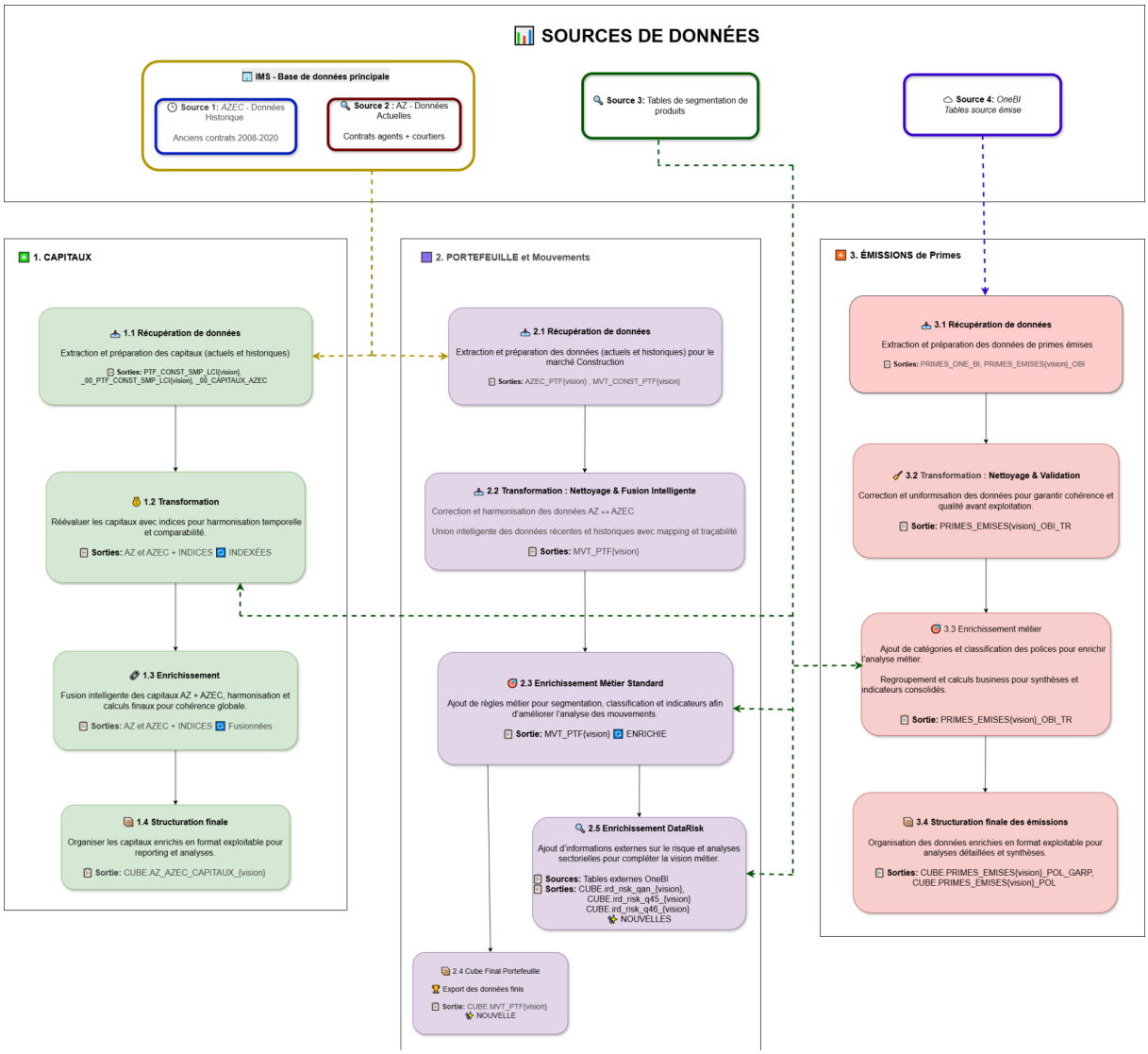


FIGURE 2.2 – Architecture détaillée des flux de données : sources, transformations par étape et tables de sortie des trois pipelines du datamart Construction

Cette architecture présente plusieurs caractéristiques techniques notables. Le **code SAS utilise massivement les macros**, mécanisme permettant de générer dynamiquement du code SAS. Certaines macros sont imbriquées sur plusieurs niveaux, rendant la lecture du code complexe et nécessitant de dérouler mentalement l'ensemble des substitutions pour comprendre la logique réelle. Les **variables globales** sont utilisées pour partager des paramètres entre différents fichiers (chemins de données, période de traitement, codes métier), mais leur portée n'est pas toujours clairement documentée. Enfin, **l'absence de documentation initiale** oblige à une lecture exhaustive du code pour comprendre l'ensemble des traitements.

Le processus d'exécution est séquentiel : les dix-neuf fichiers doivent être lancés dans un ordre précis, chaque fichier attendant que le précédent ait terminé avant de démarrer. Cette séquentialité limite les possibilités d'optimisation et allonge les temps de traitement globaux.

2.1.3 Limites de l'architecture SAS existante

Au-delà des limites stratégiques évoquées dans le chapitre un (coûts de licences, raréfaction de l'expertise, performances limitées), l'architecture SAS présente plusieurs contraintes techniques qui ont motivé la décision de migration.

— **La scalabilité est structurellement limitée** : SAS exécute les traitements sur un seul serveur de calcul. Lorsque les volumes de données augmentent ou que de nouveaux enrichissements sont ajoutés, les temps de traitement s'allongent proportionnellement. Il n'existe pas de mécanisme simple pour paralléliser les calculs ou distribuer la charge sur plusieurs machines.

— **La maintenabilité du code est coûteuse** : La complexité du code SAS, avec ses macros imbriquées et ses variables globales, rend les évolutions risquées. Lorsqu'une règle métier doit être modifiée, il faut tracer l'ensemble des endroits où cette règle intervient en tenant compte des substitutions de macros. Le risque d'effets de bord est élevé, et chaque modification nécessite des tests exhaustifs sur plusieurs visions.

— **L'absence de séparation des responsabilités** : complique le débogage et les tests. Dans l'architecture SAS, un même fichier peut mélanger la lecture des données, les filtres métier, les transformations, les jointures et l'écriture des résultats. Lorsqu'un problème survient, il est difficile d'isoler la cause exacte du dysfonctionnement.

— **La gestion des versions et de la traçabilité est rudimentaire** : Le code SAS est stocké sur des répertoires partagés, sans système de gestion de versions moderne. Les commentaires dans le code sont rares et souvent obsolètes, rendant difficile la reconstitution de l'historique des modifications.

Ces limites techniques, combinées aux enjeux stratégiques de coûts et d'expertise, ont conduit à la décision de migrer vers une architecture moderne basée sur PySpark et Azure.

2.2 Technologies modernes pour le traitement de données

2.2.1 PySpark et Apache Spark

PySpark est l'interface Python d'Apache Spark. L'API de DataFrames qu'elle expose permet de manipuler des volumes massifs de données en distribuant les calculs sur un cluster de machines, une capacité absente de l'architecture SAS d'origine. PySpark supporte nativement les opérations SQL, les agrégations complexes, les jointures et les fonctions fenêtres (*window functions*), couvrant ainsi tous les besoins de transformation du datamart Construction.

Pour ce projet, PySpark présente trois avantages décisifs : la compatibilité avec les formats modernes (Parquet, Delta, CSV), la possibilité d'exécuter le même code en local pour les tests et sur un cluster Databricks en production, et l'intégration native avec l'écosystème Azure.

2.2.2 Azure Databricks

Azure Databricks est une plateforme cloud managée qui fournit un environnement complet pour exécuter des workloads Spark : clusters gérés automatiquement, notebooks collaboratifs, intégration native avec Azure Data Lake Storage (ADLS) et support de Delta Lake. Le choix d'Azure Databricks s'aligne avec la stratégie du groupe Allianz, qui a standardisé ses infrastructures data sur Azure au niveau international (ZDNet France, 2025). Cette convergence facilite le partage de bonnes pratiques entre les entités du groupe.

2.2.3 Formats de stockage modernes

Parquet est un format de fichier binaire optimisé pour le stockage en colonnes. Il offre une compression efficace et des performances de lecture supérieures au CSV traditionnel, particulièrement pour les requêtes portant sur un sous-ensemble de colonnes. Dans le contexte du datamart Construction, le passage de CSV à Parquet permet de réduire significativement l'espace de stockage et d'accélérer les lectures.

Delta Lake, extension de Parquet, ajoute des fonctionnalités de gestion de données avancées : transactions ACID (garanties de cohérence et d'intégrité), *time travel* (accès aux versions historiques des données), et gestion de schéma avec évolution contrôlée. Ces fonctionnalités améliorent la qualité et la traçabilité des données, répondant à des exigences réglementaires croissantes dans le secteur de l'assurance.

2.2.4 L'architecture médaillon : standard de l'industrie

L'**architecture médaillon** (*medallion architecture*) est un pattern d'organisation des données popularisé par Databricks et adopté comme standard de l'industrie pour les architectures *lakehouse*. Elle organise les données en trois couches successives, chacune représentant un niveau croissant de qualité et de transformation.

Comme l'illustre la figure 2.3, les données transitent de la couche Bronze vers la couche Silver, puis vers la couche Gold, en suivant un flux unidirectionnel traité par Apache Spark et Python.

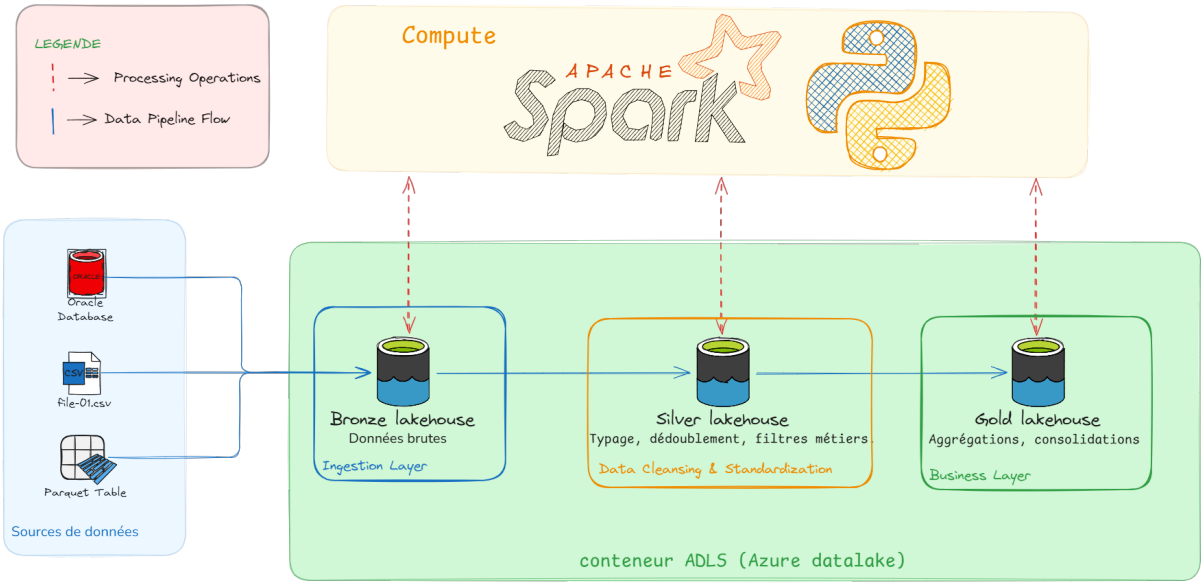


FIGURE 2.3 – Architecture médaillon implémentée : flux de données des sources vers les couches Bronze, Silver et Gold stockées dans Azure Data Lake Storage (ADLS), traitées par Apache Spark et PySpark

La **couche Bronze** constitue la zone d'ingestion brute. Les données y arrivent directement depuis les sources (fichiers CSV exportés des systèmes opérationnels, tables One BI) sans aucune transformation. Chaque fichier est stocké tel quel, partitionné par année et par mois. Cette couche est immuable : elle conserve une trace fidèle de toutes les données reçues, permettant de rejouer n'importe quel traitement depuis la source en cas de correction.

La **couche Silver** est la couche de transformation métier. Les données brutes de Bronze y sont nettoyées (correction des encodages, standardisation des types de données, gestion des valeurs nulles), filtrées selon les règles métier (marché Construction, exclusion des polices an-

nulées), et enrichies (calculs des mouvements AFN/RES/PTF, extraction des capitaux, calculs d'indexation). Le format Parquet est utilisé pour cette couche, garantissant des performances optimales pour les lectures ultérieures.

La **couche Gold** est la couche de consommation finale. Elle contient les données consolidées, prêtes pour l'analyse et la Business Intelligence. Pour le datamart Construction, la couche Gold réalise notamment la fusion des données AZ et AZEC, les enrichissements complets (IRD Risk, données clients, codes ISIC), et produit les trois sorties analytiques finaux (PTF_MVT, AZ_AZEC_CAPITAUX, PRIMES_EMISES).

Le tableau 2.1 synthétise les différences principales entre l'architecture SAS existante et l'architecture médaillon cible :

Table 2.1 – Comparaison des architectures SAS et PySpark sur les principaux critères

Critère	SAS (Existant)	PySpark (Cible)
Architecture	Monolithique, flux linéaire	Distribuée, couches Bronze/Silver/Gold
Licences	Propriétaires (coûteuses)	Open Source (gratuit)
Scalabilité	Limitée (serveur unique)	Élevée (cluster élastique Azure)
Configuration	Codée en dur dans les scripts	Externalisée (JSON/YAML)
Performance	Séquentielle, sur disque	Parallèle, en mémoire (in-memory)
Format de stockage	SAS7BDAT / CSV	Parquet optimisé (colonnaire)
Versioning du code	Absence de VCS	Git (traçabilité complète)
Maintenabilité	Macros imbriquées, difficile	Modules Python découplés et testables
Traçabilité	Minimale	Logs détaillés par couche et par étape
Reproductibilité	Partielle	Complète (rejeu depuis Bronze)

Ce tableau comparatif confirme que la migration vers PySpark et l'architecture médaillon répond directement aux limites structurelles identifiées dans l'architecture SAS existante.

2.3 Revue bibliographique et état de l'art des migrations SAS vers Python

2.3.1 Tendances de migration dans l'industrie

La migration des infrastructures SAS vers des technologies open source est une tendance de fond observée dans de nombreux secteurs, en particulier dans les services financiers et l'assurance. Les principales motivations identifiées sont la réduction des coûts de licences, la montée en popularité de Python auprès des nouvelles générations de data scientists, et l'adoption généralisée du cloud computing.

Dans le secteur de l'assurance spécifiquement, les contraintes réglementaires (Solvabilité II, IFRS 17) ont conduit les acteurs à moderniser leurs chaînes de traitement de données pour améliorer la traçabilité, l'auditabilité et la reproductibilité des calculs actuariels. L'architecture médaillon, telle que décrite par Databricks, répond précisément à ces exigences en garantissant une séparation claire entre les données brutes, les transformations intermédiaires et les résultats finaux.

2.3.2 Défis techniques connus

La littérature technique identifie plusieurs défis récurrents lors des migrations SAS vers Python. La **gestion des valeurs nulles** est l'un des plus fréquemment cités : SAS représente les valeurs manquantes numériques par un point (.) et les valeurs textuelles par une chaîne vide (""), tandis que Python/PySpark utilise la valeur `null`. Cette différence de convention, si elle n'est pas traitée explicitement, peut conduire à des erreurs silencieuses dans les calculs.

La **compatibilité des formats de dates** est un autre défi documenté : SAS stocke les dates en formats spécifiques (numéro de jours depuis le 1er janvier 1960, ou chaînes de caractères comme "14MAY1991 :00 :00 :00") qui nécessitent des fonctions de conversion explicites en PySpark. Les **macros SAS** n'ont pas d'équivalent direct en Python et doivent être refactorisées en fonctions Python paramétrables, ce qui constitue souvent la partie la plus complexe de la migration.

Enfin, la **validation de la parité fonctionnelle** est reconnue comme une étape critique et chronophage. La comparaison des résultats SAS et Python ligne par ligne, indicateur par indicateur, nécessite une méthodologie rigoureuse et un outillage adapté (Armbrust et al., 2020).

2.3.3 Bonnes pratiques identifiées

Les retours d'expérience disponibles convergent autour de plusieurs bonnes pratiques pour mener à bien une telle migration. La **documentation exhaustive du code source** avant de commencer à coder est unanimement identifiée comme un prérequis indispensable, particulièrement lorsque le code SAS n'est pas documenté initialement. Le **recensement systématique des règles métier** dans un format structuré (tableur ou base de données) permet de garantir l'exhaustivité de la migration et de fournir une base pour les tests de validation.

L'**adoption d'une architecture modulaire** facilite les tests unitaires et la maintenance. La **séparation de la configuration et du code** (externalisation des règles métier dans des fichiers JSON ou YAML) rend le code plus lisible et plus facilement évolutif. Enfin, la **validation incrémentale** (tester chaque module immédiatement après son implémentation) permet de détecter et corriger les erreurs au plus tôt, évitant l'accumulation de bugs difficiles à tracer (Zaharia et al., 2010).

Ces bonnes pratiques ont directement inspiré la méthodologie adoptée pour la migration du datamart Construction, détaillée dans le chapitre suivant.

Chapitre 3 . Méthodologie et moyens

3.1 Démarche globale et organisation du projet

3.1.1 Une approche itérative et structurée

Le projet de migration du datamart Construction a été conduit selon une **approche itérative et incrémentale**, organisée en cinq phases successives. Ce choix méthodologique s'est imposé naturellement compte tenu des contraintes du projet : un code SAS non documenté à analyser, une architecture cible à concevoir, et un critère de parité fonctionnelle strict à atteindre. Chaque phase s'est appuyée sur les livrables de la phase précédente, garantissant une progression méthodique et tracée.

Ce séquençage reflète un principe fondamental adopté dès le départ : **analyser et documenter exhaustivement avant d'écrire la première ligne de code**. Cette décision, validée avec

le maître de stage, a permis d'éviter les erreurs dues à une mauvaise compréhension de la logique métier, et d'identifier en amont les complexités susceptibles d'impacter les choix architecturaux.

Le tableau 3.1 présente la vue d'ensemble des cinq phases et leur enchaînement :

Table 3.1 – Vue d'ensemble des cinq phases de la méthodologie de migration

Phase	Intitulé	Durée	Objectif	Livrable principal
1	Analyse et documentation SAS	2 semaines	Comprendre le code source	Documentation SAS complète
2	Conception de l'architecture cible	1 semaine	Définir la structure Python	Architecture médaillon validée
3	Recensement des règles et datasets	1 semaine	Inventorier l'exhaustif	2 fichiers Excel de référence
4	Implémentation Python	5 semaines	Développer les pipelines	Code source (3 pipelines)
5	Validation et tests de parité	5 semaines	Garantir l'équivalence	Rapports de validation KPI

3.1.2 Équipe et cadre de travail

Le projet s'est déroulé au sein d'une équipe restreinte composée du stagiaire et de son maître de stage, Romeo TOHOUN, data scientist et sa responsable hiérarchique Caroline BOUCHON dans le pôle Advanced Data & Climate. Des points d'avancement hebdomadaires ont permis de valider les choix techniques, de lever les ambiguïtés métier et d'orienter les priorités de développement. La collaboration avec M. Alain VOSS, expert assurance de l'équipe, a été essentielle pour comprendre les mécanismes assurantiels sous-jacents aux règles métier codées en SAS.

3.2 Phase 1 : Analyse et documentation du code SAS existant

3.2.1 Objectifs et situation initiale

La phase d'analyse a démarré dans un contexte particulièrement contraint : dix-neuf fichiers SAS totalisant environ quinze mille lignes de code, sans documentation initiale centralisée, avec des macros imbriquées sur plusieurs niveaux et des noms de variables souvent cryptiques (`AH0`, `AMN0`, `dtechann`, etc.). L'objectif de cette phase était de comprendre en profondeur la logique métier existante avant d'entreprendre toute action de migration.

3.2.2 Démarche d'analyse systématique

L'analyse du code SAS a suivi une progression structurée, en commençant par les fichiers d'orchestration pour comprendre l'ordre d'exécution global, puis en descendant vers les macros métier qui contiennent la logique de traitement, et enfin les fichiers utilitaires spécialisés.

Les fichiers d'orchestration constituent le point d'entrée de chaque pipeline. Ils définissent les paramètres globaux (période de traitement, chemins des données, codes marchés) et appellent séquentiellement les macros métier dans l'ordre requis. Leur lecture a permis d'identifier les dépendances entre fichiers et de cartographier le flux d'exécution général.

Les macros métier constituent le cœur de la logique de traitement. Pour chaque pipeline et chaque canal de distribution (AZ et AZEC), elles encodent les filtres de sélection, les calculs des indicateurs, les jointures avec les référentiels et les règles d'agrégation. Ces macros présentent

les principales difficultés d'analyse : substitutions de variables, logique conditionnelle complexe, réutilisation de code via des paramètres dynamiques.

Le fichier `CODIFICATION_ISIC_CONSTRUCTION.sas`, avec ses vingt-cinq mille lignes de code dédiées à la classification sectorielle des risques, a constitué le défi d'analyse le plus important. Une approche ciblée sur la logique globale de fallback (*CDNAF2008* → *CDNAF2003* → *ACTPRIN*) a été privilégiée, plutôt qu'une analyse ligne par ligne, en validant la compréhension avec le maître de stage.

Pour chaque pipeline analysé, le flux de données a été systématiquement tracé en identifiant : les **sources de données** (libnames SAS et tables correspondantes), les **transformations appliquées** (filtres, calculs, jointures), et les **tables de sortie** (cubes analytiques finaux). Cette cartographie a fourni la base des schémas architecturaux présentés au chapitre 2.

3.2.3 Livrables de la phase 1

La phase d'analyse a produit une **documentation fonctionnelle SAS** complète, structurée en deux versions complémentaires.

La **version synthétique**, destinée à l'équipe, présente une vue d'ensemble des trois pipelines, des schémas de flux de données et des tableaux récapitulatifs des transformations clés. Elle constitue un document de référence pérenne, indépendant de l'environnement SAS, garantissant la capitalisation de la connaissance métier même après l'arrêt définitif du système source.

La **version technique détaillée**, destinée aux développeurs, documente chaque transformation SAS, explique les calculs complexes, cartographie les macros et leurs paramètres, et traite les cas limites identifiés lors de l'analyse. Elle a servi de référence directe pendant la phase d'implémentation Python pour vérifier la fidélité de chaque transformation.

3.3 Phase 2 : Conception de l'architecture cible Python

3.3.1 Évaluation des options architecturales

Suite à l'analyse du code SAS, deux options d'architecture ont été évaluées et présentées lors d'une réunion de cadrage avec le maître de stage.

La **réplication à l'identique** aurait consisté à reproduire la structure SAS en Python (un script Python par fichier SAS). Cette option présentait l'avantage d'être simple à valider par comparaison directe, mais elle aurait conservé l'ensemble des défauts de l'architecture monolithique source : séquentialité, couplage fort entre les modules, absence de séparation des responsabilités.

L'**architecture médaillon complète** (Bronze/Silver/Gold), standard de l'industrie tel que décrit dans le chapitre 2, a été retenue. Cette option nécessitait un effort de développement initial plus important, mais offrait les garanties de qualité, de traçabilité et d'évolutivité requises pour un système en production.

3.3.2 Principes de conception retenus

Quatre principes fondamentaux ont guidé la conception du système Python :

Configuration > Code : les règles métier qui sont susceptibles d'évoluer (filtres, mappings de colonnes, paramètres de seuils) sont externalisées dans des fichiers de configuration JSON et YAML. Cela permet de modifier le comportement du système sans toucher au code, réduisant le risque d'erreurs et simplifiant la maintenance.

Modularité : chaque composant du système remplit une responsabilité unique et bien définie. Les lecteurs de données (*readers*) sont isolés des processeurs métier (*processors*), qui sont eux-mêmes distincts des fonctions de transformation (*utils/transformations*). Cette séparation facilite les tests unitaires et la réutilisation du code entre pipelines.

Logging détaillé : chaque étape de traitement produit des logs structurés indiquant le nombre de lignes en entrée et en sortie, le temps d'exécution et les valeurs clés des indicateurs calculés. Ce niveau de traçabilité est indispensable pour diagnostiquer les éventuelles divergences lors de la validation.

Gestion d'erreurs gracieuse : les enrichissements avec des données optionnelles (référentiels parfois absents) sont traités avec des mécanismes de fallback. Un enrichissement non disponible génère un avertissement dans les logs mais ne bloque pas l'exécution du pipeline.

3.3.3 Stack technologique validée

Les choix technologiques ont été validés en cohérence avec l'infrastructure cloud d'Allianz et les recommandations de l'équipe Data Management :

Table 3.2 – Stack technologique retenue pour l'implémentation Python

Composant	Technologie	Rôle
Langage	Python 3.11+	Développement des pipelines
Framework de traitement	PySpark 3.x	Traitement distribué des DataFrames
Format de stockage	Parquet	Optimisation lectures/écritures
Configuration	JSON / YAML	Externalisation des règles métier
Versioning	Git	Traçabilité du code
Environnement de dev	ADP (tout pré-installé)	Tests et mise au point

3.4 Phase 3 : Recensement systématique des règles et datasets

3.4.1 Objectif : tout tracer avant de coder

Avant de démarrer toute implémentation, un effort systématique de recensement a été conduit pour constituer deux fichiers Excel de référence. Cette étape, inspirée des bonnes pratiques de gestion de projet data, vise à garantir l'exhaustivité de la migration en évitant les oublis qui n'auraient pu être détectés que tardivement.

3.4.2 Fichier 1 : inventaire des règles de gestion

Le premier fichier Excel (`datamart_construction_regle_de_gestion.xlsx`) recense l'ensemble des règles métier du datamart, soit **soixante-douze règles** réparties sur les trois pipelines. Chaque règle est documentée avec un identifiant unique, le pipeline concerné, une description de la règle, la référence précise au code SAS source (fichier et numéro de ligne) et l'implémentation Python correspondante.

Ce fichier a rempli un double rôle : **checklist d'implémentation** pendant la phase de développement (chaque règle étant cochée après implémentation et validation), et **base de traçabilité** permettant de vérifier après coup que chaque comportement SAS est bien couvert par le code Python.

3.4.3 Fichier 2 : recensement des datasets sources

La seconde feuille du même fichier inventorie l'ensemble des sources de données du datamart. Au total, **quarante-cinq groupes de fichiers** (*file groups*) ont été identifiés, couvrant les données mensuelles de portefeuille (fichiers IPF), les données de capitaux et garanties (POLIC_CU, CAPITXCU), les référentiels métier (segmentation, ISIC, NAF, FFB), et les données de primes (One BI). Pour chaque dataset, le fichier précise la table SAS source, le fichier SAS qui l'utilise, la version (mensuel ou référentiel) et le type (entrée source ou sortie).

Cette dernière information s'est révélée particulièrement précieuse : plusieurs datasets référencés dans le code SAS n'étaient pas disponibles dans le datalake au moment du projet (`cproduit.csv`, `prdcap.csv`, `lob.csv`, ...). Leur identification précoce a permis de les déclarer optionnels en Python et de gérer leur absence de manière gracieuse, évitant des blocages lors de l'exécution.

3.5 Phase 4 : Implémentation des pipelines Python

3.5.1 Structure du projet

L'arborescence du projet Python suit une organisation modulaire claire, reflétant la séparation des responsabilités adoptée lors de la conception :

```
project_baseline/
|-- config/                   # Configuration centralisée
|   |-- reading_config.json   # Paramètres de lecture des sources
|   |-- column_definitions.py # Schémas au format DDL pour Spark
|   |-- config.yml            # Configuration globale de l'application
|   |-- constants.py          # Constantes métier
|   |-- variables.py          # Variables globales du projet
|   `-- transformations/      # Règles métier JSON par pipeline
|-- src/
|   |-- processors/           # Logique métier par pipeline
|   |   |-- capitaux_processors/
|   |   |-- emissions_processors/
|   |   `-- ptf_mvt_processors/
|   |-- orchestrators/        # Orchestration des exécutions
|   `-- reader.py             # Lecture des données Bronze
|-- utils/
|   |-- transformations/      # Fonctions métier (base, operations)
|   |-- loaders/              # Utilitaires de chargement externes
|   |-- helpers.py            # Fonctions génériques (dates, types)
|   |-- processor_helpers.py  # Fonctions d'aide aux processors
|   `-- logger.py             # Logger structuré personnalisé
|-- docs/                     # Documentation technique (configs, workflows)
`-- main.py                   # Point d'entrée principal
```

Les **fondations du projet** (module de lecture, helpers, logger, configuration) ont été développées et testées en premier, avant le développement d'un seul pipeline métier. Cette approche a garanti que les briques de base étaient fiables avant de les utiliser dans des contextes métier plus complexes.

3.5.2 Ordre d'implémentation et stratégie de développement

L'ordre d'implémentation des pipelines a été décidé par ordre de complexité décroissante : PTF Mouvements en premier (le plus complexe, impliquant les deux canaux AZ et AZEC, la consolidation, et les enrichissements multiples), puis Capitaux (complexité intermédiaire, avec la logique d'indexation FFB), et enfin Émissions (le plus linéaire, sans consolidation AZ/AZEC).

Le choix d'aborder d'abord le pipeline le plus complexe présente un avantage stratégique majeur : les fonctions utilitaires créées pour PTF Mouvements (extraction des capitaux, calcul des mouvements AFN/RES/PTF, enrichissement ISIC, harmonisation AZ/AZEC) sont directement réutilisables pour les pipelines suivants, réduisant significativement l'effort de développement global.

À l'intérieur de chaque pipeline, une approche **itérative par transformation** a été systématiquement appliquée : coder une transformation, la tester immédiatement sur un sous-ensemble de données, logger le nombre de lignes en entrée et en sortie, corriger si nécessaire, puis passer à la transformation suivante. Cette discipline a permis de détecter et corriger les erreurs au plus tôt, avant qu'elles ne se propagent et ne deviennent difficiles à tracer.

3.5.3 Architecture des processors

Chaque pipeline est implémenté selon un pattern de classe Python orientée objet, avec trois méthodes principales : `read()` pour la lecture des données sources depuis la couche Bronze, `transform()` pour l'application des transformations métier, et `write()` pour l'écriture des résultats en couche Silver ou Gold.

Le pipeline PTF Mouvements illustre la complexité maximale de cette implémentation. Le processor AZ réalise quatorze étapes de transformation successives et indépendantes, depuis les filtres de sélection du marché Construction jusqu'à la déduplication finale. Le processor AZEC, spécifique aux entreprises de courtage construction, gère sept jointures avec des tables de référence propres à ce canal et applique une logique de calcul des mouvements différente de celle utilisée pour AZ. Le processor de consolidation fusionne ensuite les données des deux canaux, après harmonisation de leurs schémas respectifs (les colonnes portent des noms différents selon le canal : `POLICE` en AZ vs `NOPOL` en AZEC, par exemple), puis applique les enrichissements finaux (IRD Risk, données clients, codification ISIC) pour produire le cube Gold.

3.6 Phase 5 : Validation et tests de parité fonctionnelle

3.6.1 Stratégie de validation à trois niveaux

La phase de validation a été organisée selon une approche à trois niveaux de granularité croissante, inspirée des bonnes pratiques de recette fonctionnelle.

La **validation structurelle** vérifie que les fichiers de sortie Python existent, que leurs schémas contiennent les colonnes attendues avec les types corrects, et que les volumes de données sont cohérents avec les données d'entrée. Ce premier niveau permet de détecter rapidement les erreurs grossières (colonnes manquantes, types incorrects, pipeline incomplet).

La **validation fonctionnelle** compare les indicateurs agrégés (KPI) entre les sorties SAS et les sorties Python, sur plusieurs visions mensuelles. Les critères retenus sont stricts : un écart inférieur à 0,01% sur les KPI agrégés (nombre de polices, sommes de primes, totaux de capitaux) est requis pour valider la parité. Cette tolérance quasi nulle garantit que les résultats Python sont exploitables pour les analyses actuarielles et les reportings réglementaires.

La **validation par échantillonnage** compare champ par champ un échantillon de polices sélectionnées aléatoirement, permettant de détecter des erreurs compensatoires qui pourraient passer inaperçues au niveau des indicateurs agrégés.

3.6.2 Défis techniques rencontrés lors de la validation

La confrontation des résultats Python et SAS a mis en lumière plusieurs défis techniques qui ont nécessité des investigations approfondies et des corrections ciblées du code. Chaque défi a suivi le même processus de résolution : diagnostic précis par analyse du code SAS source, implémentation de la correction en Python, rejeu du pipeline sur la vision de test, et vérification de la convergence des résultats. Six problèmes principaux sont détaillés ci-dessous.

Disponibilité et accès aux données de test. L'un des défis les plus structurants du projet a été l'accès aux données nécessaires pour tester le code en continu. Les données de production du datamart Construction sont stockées dans l'infrastructure Azure d'Allianz, avec des droits d'accès stricts. Obtenir des jeux de données de test représentatifs a demandé un

temps significatif entre les demandes d'accès, les validations par les équipes sécurité, et la mise à disposition effective des fichiers. Cette contrainte a imposé de travailler parfois « à l'aveugle » sur certaines transformations, en les testant uniquement une fois les données disponibles, ce qui a parfois allongé les cycles de détection et de correction des erreurs.

Colonne CDPOLE absente pour le canal AZEC. Lors de la consolidation des données AZ et AZEC, une divergence frappante est apparue : le nombre de polices avec `cdpole = 3` (courtage) était significativement inférieur en Python par rapport à SAS. L'analyse du code SAS source (`PTF_MVTS_CONSOLIDATION_MACRO.sas`) a révélé que SAS assigne explicitement la valeur `"3"` à la colonne `cdpole` lors de la consolidation AZEC via une instruction littérale dans la macro. Le processor AZEC Python créait bien la colonne `dircom` (canal de distribution) mais omettait de créer `cdpole`, laissant seize mille lignes avec une valeur nulle. L'ajout d'une seule instruction dans le processor AZEC a suffi à rétablir la parité complète sur cet indicateur.

Traitement de la colonne `dtechann` pour l'indexation. La colonne `dtechann`, qui représente la date d'échéance annuelle d'une police, est stockée dans les fichiers sources sous forme de chaîne de caractères avec un format variable selon l'ancienneté de la donnée : certaines polices historiques utilisent un format à quatre chiffres (MMJJ), d'autres un format à six chiffres (MMJJAA), et d'autres encore un format complet. Plutôt que de tenter une conversion vers un type date natif, opération qui échouait sur les cas non standards, la solution retenue a consisté à lire cette colonne systématiquement en type `String` et à reproduire en Python la logique SAS : extraire le mois et le jour par découpage de la chaîne en tenant compte du nombre de caractères, puis reconstituer la date dans un format normalisé. Cette approche, fidèle à la façon dont SAS traite ces données, a permis de couvrir l'intégralité des cas sans exception.

Dates au format SAS datetime pour les enrichissements IRD. Les trois tables d'enrichissement IRD Risk (Q45, Q46, QAN) contiennent des dates stockées au format SAS datetime : une chaîne du type `'14MAY1991:00:00:00'` qui ne correspond à aucun format standard reconnu par la fonction `to_date()` de Spark. Chaque tentative de conversion directe produisait des valeurs nulles, ce qui annulait silencieusement les jointures d'enrichissement. La solution a consisté à développer un helper dédié (`parse_sas_datetime()`) qui extrait la partie date de la chaîne (le segment situé avant le premier séparateur `:`) et la convertit ensuite vers un format standard. Ce type de problème, invisible à la compilation car il ne génère pas d'erreur Python, ne pouvait être détecté que par la comparaison des résultats avec le SAS sans compter que plusieurs autres tables présentaient des problèmes similaires avec des formats différentes.

Structure de configuration des filtres pour le pipeline Émissions. Le pipeline Émissions applique plusieurs catégories de filtres d'exclusion (intermédiaires exclus, garanties exclues, produits hors périmètre, etc.) dont les valeurs sont stockées dans des fichiers de configuration JSON. Ces configurations avaient été structurées sous la forme d'objets enrichis `{"description": "...", "values": [...]}` pour faciliter la documentation. Or, lorsque Spark tentait de convertir directement cet objet en littéral pour une clause `isin()`, il générait une erreur : Spark ne sait pas comment interpréter un dictionnaire comme une liste de valeurs. La correction a consisté à extraire explicitement la liste `values` avant d'en créer le littéral Spark, sans modifier la structure de configuration qui reste plus lisible avec sa description.

Harmonisation des schémas AZ et AZEC. La consolidation des données des deux canaux de distribution suppose que leurs colonnes soient compatibles pour un `UNION`. Or, plusieurs colonnes portent des noms différents selon le canal d'origine : la clé de police s'appelle `POLICE` en AZ et `NOPOL` en AZEC, l'intermédiaire s'appelle `INTERMED` d'un côté et `NOINT` de l'autre, et ainsi de suite pour une dizaine de colonnes. Reproduire manuellement ces mappings dans le code Python aurait rendu le code rigide et difficile à maintenir. La solution retenue a consisté à externaliser l'ensemble de ces correspondances dans un fichier de configuration JSON, permettant d'appliquer automatiquement tous les renommages nécessaires avant la consolidation.

3.7 Outils, environnement et livrables finaux

3.7.1 Environnement de développement

Le développement a été réalisé dans un environnement cloud ADP (Allianz Data platform) qui est automatiquement connecté à azure via un token, permettant de tester l'intégralité des pipelines sans nécessiter de connexion ardure à l'infrastructure cloud Azure. Cette autonomie a facilité le cycle de développement itératif, réduisant les temps de feedback entre la modification et le test. La migration vers Azure Databricks est prévue pour la phase de déploiement en production, postérieure à la période du stage.

La gestion du code source a été assurée via Git, avec un dépôt versionné permettant de tracer l'ensemble des modifications et de revenir en cas de régression. Cette pratique, absente du système SAS source, constitue l'une des améliorations fondamentales apportées par la migration.

3.7.2 Synthèse des livrables produits

L'ensemble des travaux du stage a produit quatre catégories de livrables :

Synthèse des livrables produits lors du stage

1. **Documentation Fonctionnelle & Analyse SAS**

— `general_sas_architecture.drawio` — *Architecture générale SAS (datamart).*

— `sas_detail_flow_architecture.drawio` — *Architecture détaillée des trois workflows SAS.*

— `documentation_datamart_construction_sas_26112025.pptx` — *Présentation complète (logique métier, traitements, organisation).*

— `Architecture_Générale_doc_sas.docx` — *Explication de la documentation générale du code SAS.*

— `Analyse de l'Architecture SAS Construction.md` — *Documentation détaillée (Markdown).*

2. **Documentation Métier (Règles & Datasets)**

— `datamart_construction_regle_de_gestion.xlsx` — *72 règles métier + 45 datasets inventoriés.*

3. **Développement Python**

— `python_architecture.excalidraw` — *Architecture Medallion (Bronze → Silver → Gold), modules et pipelines.*

— *Pipelines Python alignés avec SAS (dépôt GitHub interne Allianz).*

4. **Assurance Qualité (QA)**

— `kpis_results.xlsx` — *Comparaisons SAS vs Python (KPI, écarts, performances).*

Chapitre 4 . PRÉSENTATION ET ANALYSE DES RÉSULTATS

4.1 Architecture finale implémentée

L'architecture Médaillon définie en phase de conception a été intégralement déployée dans Azure Data Lake Storage selon la hiérarchie suivante :

```
{conteneur_datalake}/
|-- bronze/
|   |-- {YYYY}/{MM}/    <- données mensuelles (ipf16.csv, capitaux, primes...)
|   `-- ref/            <- référentiels stables (ISIC, NAF, FFB, IRD Risk, SIRET)
|-- silver/
|   `-- {YYYY}/{MM}/    <- résultats intermédiaires par pipeline
`-- gold/
    `-- {YYYY}/{MM}/    <- cubes finaux consommables par la BI
```

Le partitionnement temporel (`{YYYY}/{MM}/`) permet un traitement incrémental par vision mensuelle, en s'appuyant sur quarante-trois *file groups* configurés avec des schémas Spark stricts.

4.2 Résultats par pipeline

Au total, l'architecture produit **sept fichiers Delta Parquet distincts** par vision mensuelle, répartis entre les tables Silver (intermédiaires) et Gold (consolidations finales).

Fichiers de sortie produits par vision mensuelle

- **PTF Mouvements**
  - **Silver** : `ptf_mvt_{vision}`
  - **Silver** : `ird_risk_q45_{vision}`
  - **Silver** : `ird_risk_q46_{vision}`
  - **Silver** : `ird_risk_qan_{vision}`
- **Capitaux**
  - **Gold** : `az_azec_capitaux_{vision}`
- **Émissions**
  - **Gold** : `primes_emises_{vision}_pol`
  - **Gold** : `primes_emises_{vision}_pol_garp`

4.2.1 Pipeline PTF Mouvements

Le pipeline PTF Mouvements consolide les mouvements de portefeuille AZ et AZEC, applique les règles de filtrage métier, puis génère la table Gold utilisée pour les analyses BI. Sur l'ensemble des visions testées, les volumes Python et SAS sont strictement identiques, aussi bien en nombre total de polices qu'en répartition par pôle et par code produit.

La vision 202512 sert d'exemple illustratif. La distribution par pôle obtenue est identique entre les deux systèmes :

Table 4.1 – Distribution du portefeuille par pôle (exemple : vision 202512)

Pôle	Canal	Python	SAS	Écart
Pôle 1 (Agent)	AZ	32 319	32 319	0,00%
Pôle 3 (Courtage)	AZ + AZEC	54 693	54 693	0,00%
Total		**87 012**	**87 012**	**0,00%**

4.2.2 Pipeline Capitaux

Le pipeline Capitaux applique l'indexation financière FFB et consolide les données AZ et AZEC dans une table unique. Sur toutes les visions testées, les volumes Python correspondent exactement aux volumes SAS et les indicateurs métier (`VALUE_INSURED`, `RISQUE_DIRECT`) présentent une parité totale.

4.2.3 Pipeline Émissions

Le pipeline Émissions applique les règles métier sur les primes issues de One BI puis génère deux tables Gold (par police et par garantie). Sur l'ensemble des visions testées, les volumes et indicateurs Python (`Primes_N`, `Primes_X`, `MTCOM_X`) sont strictement identiques à la référence SAS.

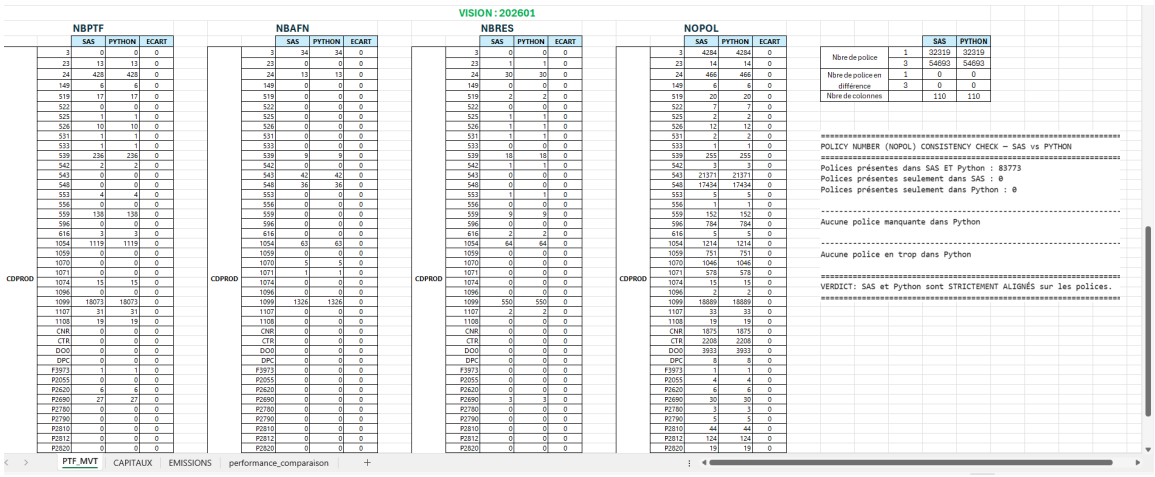
4.3 Validation de la parité fonctionnelle

La validation a été conduite sur plusieurs visions mensuelles et porte sur les trois pipelines. Elle s'appuie sur trois niveaux de contrôle :

- **Structure** : colonnes, types, schémas Spark, présence des champs obligatoires.
- **Volumes** : comparaison du nombre de polices par canal, par pôle et par code produit.
- **KPI métier** : comparaisons détaillées des indicateurs SAS vs Python.

L'ensemble des résultats détaillés est centralisé dans le fichier `kpis_results.xlsx`. Chaque feuille correspond à un pipeline et à une vision mensuelle, et documente les écarts éventuels (colonnes `ECART`) ainsi que les contrôles de cohérence par numéro de police.

La figure suivante illustre la structure du fichier :



Sur toutes les visions testées, les indicateurs calculés par Python sont **strictement identiques** à ceux obtenus par SAS : aucun écart n'est constaté, que ce soit sur les volumes, les distributions ou les KPI métier. Cette parité confirme la fidélité de l'implémentation Python par rapport au comportement SAS.

4.4 Analyse de la structure du code Python produit

4.4.1 Modularité et réutilisabilité

Le code Python a été structuré de manière modulaire afin de faciliter sa maintenance et sa réutilisation dans d'autres marchés du programme SAS EXIST. Le module `src/reader.py` est générique et lit automatiquement les file groups définis dans `reading_config.json`, tandis que les transformations sont factorisées dans `utils/transformations/`.

4.4.2 Configuration externalisée

Les paramètres métier (file groups, règles de filtrage, mappings, schémas) sont intégralement externalisés dans des fichiers JSON et des définitions Spark centralisées. Cela permet d'adapter le comportement du pipeline sans modifier le code Python, ce qui améliore fortement la maintenabilité.

4.4.3 Comparaison qualitative SAS vs Python

Comparaison qualitative : SAS vs Python

- **Versionnement et documentation**
  - SAS : versionnement manuel, documentation limitée.
  - Python : version Git complet, documentation technique + fonctionnelle.
- **Modularité**
  - SAS : macros monolithiques difficilement extensibles.
  - Python : modules isolés, configuration externalisée.
- **Traçabilité et testabilité**
  - SAS : logs peu détaillés, tests complexes.
  - Python : logs structurés, contrôles automatiques, exécution distribuée via Spark.

Chapitre 5 . DISCUSSION ET PERSPECTIVES

5.1 Analyse critique du projet

Ce qui a bien fonctionné

Le résultat le plus concret est simple : les trois pipelines tournent, les sorties PTF et Capitaux sont identiques au SAS, et une documentation complète existe désormais. Aucun de ces trois points n'était acquis au départ.

Le choix d'**analyser avant de coder** a été déterminant. Les deux semaines initiales consacrées à la lecture du code SAS ont évité de nombreuses erreurs d'interprétation qui auraient été difficiles à rattraper en fin de projet. Sur une migration sans documentation initiale, c'était indispensable.

L'**architecture médaillon** a aussi été un vrai avantage pendant la validation. Comme chaque couche est persistée, il est possible de vérifier les données étape par étape et d'identifier précisément l'origine d'une divergence. Sans cette séparation, déboguer la correction `cdpole` aurait été beaucoup plus complexe.

Enfin, la **configuration externalisée** s'est révélée très pratique : corriger un filtre ou ajouter un code produit ne nécessite pas de modifier le code Python, seulement un fichier JSON. C'est l'opposé de SAS, où les paramètres étaient éparpillés dans le code.

Ce qui a été difficile

La difficulté principale a été **l'absence totale de documentation SAS**. Toute la logique se trouvait dans des macros imbriquées, avec des variables peu explicites. Cela a ralenti le démarrage et laisse toujours un risque résiduel : certaines règles implicites peuvent exister sans être visibles dans le code.

L'**accès aux données de test** a également été un défi. Les règles de sécurité Azure d'Allianz sont strictes et l'obtention d'extraits réels pouvait prendre du temps. Plusieurs transformations ont dû être codées sans données réelles, puis ajustées une fois l'accès obtenu.

5.2 Positionnement par rapport aux pratiques de l'industrie

La solution développée est cohérente avec les standards actuels du data engineering. L'architecture médaillon est la recommandation de référence pour les architectures *lakehouse*, et le choix d'Azure Databricks correspond à la stratégie cloud d'Allianz.

Par rapport à d'autres approches observées dans l'assurance, notamment celles basées sur Pandas qui atteignent vite leurs limites, PySpark permet de traiter des volumes bien supérieurs. La structure modulaire du code, conforme aux conventions Python, marque aussi une vraie rupture avec le SAS d'origine.

5.3 Ce que ça apporte à Allianz France

Trois bénéfices concrets ressortent :

— **Réduction des coûts** : la migration permet à terme de réduire les licences SAS.

— **Scalabilité** : Spark s'adapte automatiquement aux volumes croissants, ce qui n'est pas le cas de l'environnement SAS actuel.

— **Capitalisation documentaire** : avant le projet, la connaissance du datamart Construction était uniquement dans le code SAS. Elle est maintenant formalisée dans plusieurs documents utilisables par toute l'équipe.

Deux points de vigilance se dégagent également : la période de coexistence SAS/Python, qui a un coût opérationnel, et la montée en compétences PySpark des équipes, qui doit être accompagnée.

5.4 Perspectives

Court terme : finaliser la validation des dix visions historiques pour les pipelines Capitaux et Émissions. Le pipeline PTF Mouvements est déjà stabilisé.

Moyen terme : réutiliser le framework développé (module de lecture, logger, configuration JSON, structure de projet) pour les quatre marchés suivants : Auto, Habitation, Santé et Épargne. Les prochaines migrations pourront se concentrer sur les règles métier spécifiques plutôt que sur les briques techniques.

Long terme : lorsque tous les marchés auront été migrés, l'infrastructure SAS pourra être désactivée. Une évolution naturelle sera l'utilisation avancée de Delta Lake (ACID, time travel) pour les marchés futurs.

CONCLUSION

Ce stage de six mois au sein du pôle Advanced Data & Climate d'Allianz France a permis de mener à bien la première migration opérationnelle d'un datamart SAS vers une architecture Python/PySpark en contexte industriel. Conçue comme un prototype pilote du programme SAS EXIST, cette mission dépasse largement le simple exercice technique : elle constitue à la fois une démonstration de faisabilité, une méthodologie formalisée et un ensemble de livrables directement réutilisables pour les migrations à venir.

Sur le plan technique, l'objectif principal — garantir la parité fonctionnelle avec le système SAS — a été atteint pour les pipelines PTF Mouvements et Capitaux. Les comparaisons réalisées sur la vision de référence montrent un alignement strict : mêmes volumes de lignes, mêmes répartitions par pôle et par produit, même structure de colonnes. La démarche itérative adoptée, fondée sur cinq phases clairement définies, une validation continue et une résolution des divergences par investigation systématique, a permis d'identifier et de corriger des différences parfois subtiles qui n'auraient pas été détectées par une simple comparaison des totaux. La migration ne s'est donc pas limitée à « traduire » du SAS vers Python : elle a permis de réarchitecturer entièrement le datamart selon les principes de l'architecture médaillon (Bronze/Silver/Gold), en externalisant la configuration et en produisant un code modulaire pensé pour durer.

Sur le plan méthodologique, ce stage a mis en évidence les conditions réellement nécessaires à une migration de ce type. L'absence de documentation du code SAS d'origine a constitué le premier obstacle : comprendre et reconstituer les règles métier à partir de macros imbriquées a représenté une part importante de l'effort global. Ce constat souligne l'importance de la documentation fonctionnelle produite dans ce prototype, qui servira de base pour les prochaines migrations. De même, la disponibilité des données de test en environnement Azure s'est révélée être un facteur critique : les protocoles de sécurité stricts ont entraîné des délais non anticipés, rappelant qu'une migration en contexte industriel implique nécessairement des contraintes opérationnelles fortes.

Pour Allianz France, les bénéfices attendus s'articulent autour de trois axes. Le premier est économique, avec la perspective de réduire progressivement les coûts liés aux licences SAS. Le deuxième est technique : le traitement distribué sur Azure Databricks offre une scalabilité bien supérieure à celle de l'environnement SAS historique. Enfin, la capitalisation documentaire produite dans le cadre de ce stage constitue un actif durable, en rendant explicite une connaissance métier qui était jusque-là intégralement noyée dans quinze mille lignes de code SAS non maintenu.

Les perspectives ouvertes par ce prototype sont immédiates. Le programme SAS EXIST prévoit la migration de quatre autres marchés (Auto, Habitation, Santé, Épargne). Le framework développé — lecteur générique, infrastructure de logging, configuration externalisée, structure de projet et méthodologie de validation — est conçu pour être réutilisé sans modification majeure. La prochaine étape naturelle consiste à finaliser la validation de parité sur dix visions historiques, puis à engager la migration du deuxième datamart en capitalisant sur l'ensemble des enseignements de ce premier cycle.

BIBLIOGRAPHIE

Documents de presse et rapports institutionnels

— **Wikipédia**, 2026. Allianz : présentation générale du groupe Allianz, histoire, organisation, gouvernance et données institutionnelles (fiche institutionnelle). Disponible sur : https://fr.wikipedia.org/wiki/Allianz

— **Wikipédia**, 2026. Allianz France : historique, organisation, chiffres clés et évolution de la filiale française du groupe Allianz (fiche institutionnelle). Disponible sur : https://fr.wikipedia.org/wiki/Allianz_France

— **ZDNet France**, 2026. Allianz France change de DSI pour accélérer la transformation technologique : annonce managériale et stratégique (article de presse). Disponible sur : https://www.zdnet.fr/actualites/allianz-france-change-de-dsi-pour-accelerer-la-transformation-technologique-490285.htm

— **Allianz France — Newsroom**, 2025. Baromètre des risques d'Allianz 2025 : incidents cyber en tête, changement climatique à son plus haut niveau : analyse annuelle des risques majeurs identifiés par plus de 3 700 experts dans 100 pays (rapport institutionnel). Disponible sur : https://newsroom.allianz.fr/barometre-des-risques-dallianz-2025-incidents-cyber-en-tete-changement-climatique-a-son-plus-niveau/

— **Fédération Française du Bâtiment**, 2025. Indice FFB du coût de la construction : référence des coûts utilisés en assurance construction (données institutionnelles). Disponible sur : https://monimmeuble.com/actualite/indice-ffb-du-cout-de-la-construction-2025

— **Fédération Française du Bâtiment**, 2024. Indices et index du bâtiment (BT, ICC FFB) : indicateurs économiques du secteur du bâtiment (données institutionnelles). Disponible sur : https://www.ffbatiment.fr/outils-modeles-document/outils/indices-et-index-batiment

— **INSEE**, 2025. Indices de coûts et de prix dans la construction : statistiques trimestrielles utilisées en analyse actuarielle (données institutionnelles). Disponible sur : https://www.insee.fr/fr/statistiques/2015347

Documentation technique officielle

— **Apache Software Foundation**, 2024. PySpark API Reference — Apache Spark 3.5.0 : documentation API officielle (documentation technique). Disponible sur : https://spark.apache.org/docs/latest/api/python/index.html

— **Apache Software Foundation**, 2024. Apache Spark — Unified Analytics Engine : présentation générale du framework Spark (documentation technique). Disponible sur : https://spark.apache.org/

— **Databricks**, 2024. Medallion Architecture : A data design pattern for the Lakehouse : description du modèle Lakehouse (documentation technique). Disponible sur : https://www.databricks.com/glossary/medallion-architecture

— **Delta Lake Project**, 2024. Delta Lake vs Parquet Comparison : analyse détaillée des différences entre Parquet et Delta, incluant structure, performances et cas d'usage (article technique). Disponible sur : https://delta.io/blog/delta-lake-vs-parquet-comparison/

— **Databricks**, 2024. Delta Lake Documentation : documentation officielle du moteur Delta Lake (documentation technique). Disponible sur : https://docs.delta.io/latest/index.html

— **SAS Institute Inc.**, 2023. SAS 9.4 Language Reference : Concepts : description des éléments clés du langage SAS (documentation technique). Disponible sur : https://documentation.sas.com/doc/en/pgmsascdc/9.4_3.4/lesysoptsref/titlepage.htm

— **SAS Institute Inc.**, 2023. SAS Help Center — SAS 9.4 and SAS Viya 3.5 : base documentaire générale SAS (documentation technique). Disponible sur : https://support.sas.com/documentation/

— **Microsoft Corporation**, 2024. Azure Databricks documentation : documentation officielle de la plateforme Azure Databricks (documentation technique). Disponible sur : https://learn.microsoft.com/en-us/azure/databricks/

— **Microsoft Corporation**, 2024. Introduction to Azure Data Lake Storage Gen2 : description du service ADLS Gen2 (documentation technique). Disponible sur : https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-introduction

Documents internes Allianz France

— **Allianz France — Pôle Advanced Data & Climate**, 2025. Documentation du datamart Construction SAS : présentation fonctionnelle (document interne). Auteur : Stéphane KPOVIESSI.

— **Allianz France — Pôle Advanced Data & Climate**, 2025. Architecture_Générale_doc_sas : document d'architecture SAS (document interne). Auteur : Stéphane KPOVIESSI.

— **Allianz France — Pôle Advanced Data & Climate**, 2025. Analyse de l'Architecture SAS Construction : description des règles métier du datamart Construction (document interne). Auteur : Stéphane KPOVIESSI.

— **Allianz France — Pôle Advanced Data & Climate**, 2026. datamart_construction_regle_de_gest recensement des règles de gestion et sources du datamart Construction (document interne). Auteur : Stéphane KPOVIESSI.

— **Allianz France — Pôle Advanced Data & Climate**, 2025–2026. Dépôt Git interne du datamart Construction : code source et documentation technique (document interne). Auteur : Stéphane KPOVIESSI.

Normes et standards

— **Python Software Foundation**, 2013. PEP 8 — Style Guide for Python Code : guide de style officiel du langage Python (standard technique). Disponible sur : https://peps.python.org/pep-0008/

— **JUNIA**, 2024. Charte de l'écrit : exigences rédactionnelles pour les rapports écrits (document pédagogique interne).